

Team preparation for the SIH technical review

1. Problem We Are Solving

Our platform provides trusted identity, lifecycle management and verification for physical and digital assets using blockchain, while maintaining Web2-level performance through an off-chain indexer.

Blockchain is used for trust, provenance and verification. Do not claim that blockchain by itself proves a physical product is genuine; physical authenticity depends on the registration and verification process.

2. Architecture

Blockchain Layer: Solidity smart contract, ERC-721, OpenZeppelin AccessControl, `MANAGER_ROLE`, mint/transfer/revoke and blockchain events.

Identity & Security: SIWE/EIP-4361, wallet signatures, nonce, did:ethr and JWT sessions.

Backend: Python, FastAPI, Web3.py, SQLAlchemy, SQLite and the event indexer.

Frontend: React + Vite, Ethers.js, MetaMask, management dashboard and public `/verify/`:serial portal.

3. Hybrid Architecture — Most Important

Blockchain = trust. SQL = speed.

The blockchain is the authoritative source of truth. The event indexer listens for blockchain events and synchronizes relevant information into SQL. The frontend queries SQL for fast application operations instead of repeatedly querying the blockchain.

4. Innovation — What Makes Our Solution Different

Our innovation is not simply “using blockchain.” It is the way the components are combined to solve the trust-versus-performance problem.

1. Hybrid Blockchain + SQL Indexing: Blockchain provides trusted, tamper-resistant asset history, while the Python event indexer converts on-chain events into a fast, queryable SQL layer. This gives blockchain-level trust with Web2-style application performance.

2. DID Interoperability: We use **did:ethr**, giving users a standardized decentralized identity representation rather than locking identity to only our database. This creates potential interoperability with other decentralized systems.

3. Cryptographic Audit Trail: Sensitive asset lifecycle actions are authorized through the on-chain **MANAGER_ROLE** and recorded as blockchain transactions/events. This provides a verifiable record of who authorized important lifecycle changes.

4. Practical Web3 UX: MetaMask integration through `wallet_watchAsset` reduces the friction of viewing registered assets, while `localStorage` deduplication prevents repeated import prompts.

5. Public Verification: The `/verify/`:serial portal allows external users to verify an asset without needing an account.

Best reviewer answer: “Our innovation is using each technology for what it does best: blockchain for trust and provenance, decentralized identity for interoperability, SQL indexing for performance, and a normal web interface for usability.”

5. Event Indexer

The indexer is a background Python service that watches blockchain events. Flow: Smart Contract → Blockchain Event → Python Indexer → SQL Database → Frontend.

One-line answer: “The indexer acts as a bridge between the blockchain's trusted event history and our application's fast query layer.”

6. ERC-721 & Smart Contract

ERC-721 is used because our assets are unique and individually identifiable. The smart contract contains the asset rules on-chain. OpenZeppelin AccessControl restricts sensitive operations using `MANAGER_ROLE`.

Key line: "The authorization is enforced by the smart contract, not just by hiding buttons in the frontend."

7. SIWE, Nonce, DID & JWT

SIWE: Users prove wallet ownership by signing a server-generated authentication message instead of using a password.

Nonce: A unique value that prevents an old login signature from simply being reused.

did:ethr: A standardized decentralized identity representation linked to the Ethereum address.

JWT: A short-lived session token issued after successful wallet authentication, used for subsequent API requests.

8. MetaMask

MetaMask provides the user's wallet interface for signing authentication messages and interacting with the blockchain. `wallet_watchAsset` is used to request asset display/import, while `localStorage` deduplication prevents repeated import prompts.

9. Public Verification

The `/verify/:serial` route allows anyone to enter an asset serial number and verify its registered status without authentication. This can serve customers, auditors, partners and external organizations.

10. Orphaned Assets & Deleted Users

Deleting a database user does not automatically delete blockchain assets. Before deletion, the backend checks the blockchain using `getAssetsByOwner` so assets can be handled/revoked first.

For deleted users, `User.is_active` is checked during authentication so inactive users cannot continue authenticating through the system.

11. Why SQLite / FastAPI / Web3.py

SQLite: Lightweight and suitable for the prototype; production can migrate to PostgreSQL.

FastAPI: Efficient Python backend with asynchronous support.

Web3.py: Lets the Python backend communicate with the Ethereum-compatible blockchain and smart contract.

Ethers.js: Lets the React frontend interact with the wallet/blockchain.

12. Local Blockchain & Scalability

Chain ID 31337 is the local Ethereum-compatible development network used by the prototype. The architecture can be adapted to a testnet or production network.

For production: use a production database, dedicated indexer service, checkpoints, retries, reconciliation, monitoring and security hardening.

13. Business Scope

Potential revenue streams: B2B SaaS subscriptions, verification API usage fees, and asset registration/minting fees.

Potential markets: manufacturing, logistics, luxury/high-value goods, insurance and institutional identity.

14. Important Reviewer Answers

Why blockchain? For tamper-resistant, independently verifiable asset provenance.

Why ERC-721? Because each asset is unique.

Why indexer? Blockchain gives trust; SQL gives query performance.

Source of truth? Blockchain; SQL is a derived index.

Why SIWE? Passwordless wallet authentication.

Why JWT? Maintain the authenticated application session.

What's innovative? Combining blockchain trust, decentralized identity and Web2-level query performance.

15. If You Don't Know a Technical Term

Do not bluff. Say: "I'm familiar with how we're using it in our architecture, although I may not know the protocol-level details." Then explain its role in our system.

16. Team Contribution

Initial stage: each member independently explored/prototyped approaches. Then the team compared ideas and selected useful concepts. The final prototype was integrated into one system, with the primary implementation handled by the member who built the final prototype. Describe everyone's actual contributions honestly.

17. 30-Second Project Explanation

"Our platform provides trusted identity and lifecycle management for physical and digital assets. We represent unique assets using ERC-721 smart contracts on an Ethereum-compatible blockchain, with role-based authorization for sensitive operations. Users authenticate through Sign-In With Ethereum, and we use decentralized identifiers for identity representation. Our event indexer listens to blockchain events and synchronizes them into a SQL database, allowing blockchain-level trust with fast Web2-style queries. The React frontend integrates MetaMask, while a public verification portal allows anyone to verify an asset using its serial number."

18. Final Rules

DO: Explain why each technology exists, understand the complete architecture, use simple language first, be honest about limitations, and let the person who knows a component best handle deep questions.

DON'T: Claim blockchain automatically proves physical authenticity, say SQL is the source of truth, claim the prototype is fully production-ready, use buzzwords you cannot explain, or claim features that are not implemented.